

DATA VERSIONING

✓ DATA MANAGEMENT

Data Management means collecting, storing, organizing, and protecting data so it can be used easily and correctly.

Why is it important?

- Keeps data accurate and safe
- Helps businesses make better decisions
- Makes finding information faster and easier
- Supports reporting, analytics, and AI systems

Simple Example

A shopping app stores customer orders and payment details. Data management helps keep this information organized, updated, and secure.

✓ DATA DRIFT

Data drift happens when the **new real-world data becomes different from the data used to train the machine learning model.**

Because of this:

- Model accuracy starts decreasing
- Predictions become less reliable

SCHEMA-ON-WRITE

Schema-on-write is a data management approach where data structure and validation rules are enforced before data is stored.

Example

Before inserting data into a SQL database:

- Column names
- Data types
- Relationships

must already be defined.

If the data does not match the schema:

- The write operation fails.

SCHEMA-ON-READ

Schema-on-read is a flexible data architecture approach where schema interpretation occurs during data retrieval rather than during storage. Raw data is stored first.

Example

A data lake stores:

- JSON logs
- CSV files
- Clickstream events

without enforcing strict structure initially.

When analysts query the data:

- The schema is interpreted dynamically.

EVENT SOURCING

Event sourcing is an architectural pattern where application state changes are stored as an immutable sequence of events.

✓ **IMPORTANCE OF GIVING ONLY READ-ONLY PERMISSION TO RAW DATA**

Giving write permissions to raw data in ML systems is a bad idea because raw data should remain original and unchanged.

Problems with write access:

- Data can be accidentally modified or deleted
- Incorrect data can enter the system
- Model training results become unreliable
- It becomes difficult to trace errors
- Security and compliance risks increase

Best Practice

Raw data should usually be **read-only**.

Any cleaning or transformation should happen on a separate processed copy of the data.

✓ **SLOWLY CHANGING DIMENSIONAL (SCD) DATA**

In Slowly Changing Dimension (SCD) data, the **dimension data changes slowly over time**, not very frequently.

This usually includes descriptive information such as:

- Customer address
- Phone number
- Employee department
- Product category

Example

A customer changes their city from Delhi to Mumbai.

The customer exists already, but one of their details changes slowly over time.

SCD techniques are used in data warehouses to track and manage these changes properly.

✓ **STARTUPS CHOOSE NON-RELATIONAL DATABASE**

Start-ups prefer non-relational databases (NoSQL) in early stages because they are:

- **Flexible** → Easy to change data structure as requirements change
- **Fast to develop** → No fixed schema needed
- **Scalable** → Can handle growing users and data easily
- **Cost-effective** → Good for rapid development and scaling
- **Better for unstructured data** → Useful for apps with changing or varied data

Example

A new social media app may frequently change features and user data formats. NoSQL databases make these changes easier without redesigning the whole database.

✓ **WHY AND WHEN TO USE RELATIONAL DATABASE**

Relational databases are preferred for business use cases that require:

- **High data accuracy**
- **Structured data**
- **Strong relationships between data**
- **Transactions and consistency**

Common Use Cases

- Banking systems
- Payment applications
- E-commerce order management
- Inventory systems

- Airline and hotel bookings
- ERP and CRM systems

Why?

Relational databases support:

- ACID transactions
- Data integrity
- Foreign key relationships
- Reliable querying using SQL

Example

In banking, money transfers must be accurate and consistent. Relational databases ensure that transactions are completed correctly without data loss.

✓ CAP THEOREM

CAP Theorem states that a distributed database system can provide only **two out of these three properties at the same time**:

1. **Consistency (C)** → All users see the same latest data
2. **Availability (A)** → The system always responds to requests
3. **Partition Tolerance (P)** → The system continues working even if network failures occur

Simple Idea

When a network problem happens, a system must choose between:

- keeping data fully consistent, or
- keeping the system fully available.

Example

Many NoSQL databases choose:

- **AP** → Availability + Partition Tolerance
Some relational/distributed systems prefer:
- **CP** → Consistency + Partition Tolerance.

✓ DATA LINEAGE

Data lineage is the process of tracking where data comes from, how it changes, and where it moves across systems.

Importance for Debugging

- Helps find where errors started
- Tracks incorrect data transformations
- Makes troubleshooting faster
- Improves data trust and accuracy

Example

If a report shows wrong sales numbers, data lineage helps trace the issue back to the exact database, pipeline, or transformation step causing the problem.

✓ OLAP vs OLTP

OLTP (Online Transaction Processing)

Used for handling daily transactions like banking, orders, and payments.

Focuses on fast insert, update, and delete operations.

OLAP (Online Analytical Processing)

Used for data analysis, reporting, and business intelligence.

Focuses on complex queries and large-scale data analysis.

Priorities

- **OLTP:** Speed, accuracy, concurrency, data consistency
- **OLAP:** Fast analytics, query performance, historical insights

Databases

OLTP Databases

- [MySQL](#)
- [PostgreSQL](#)
- [Oracle Database](#)

OLAP Databases

- [Snowflake](#)
- [Google BigQuery](#)
- [Amazon Redshift](#)

✓ DUCKDB

[DuckDB](#) is an in-process analytical database designed for fast OLAP queries on large datasets.

Why do we need DuckDB?

- Fast analytics on local files like CSV and Parquet
- Lightweight and easy to use
- No separate server setup required
- Works well with Python and data science tools
- Good for data analysis on laptops or small systems

Simple Example

A data analyst can directly query a large CSV file using SQL without importing it into a traditional database.

✓ DATA MART, DATA WAREHOUSE, DATA LAKE, DATA LAKEHOUSE

1. Data Mart

A Data Mart is a smaller, department-specific subset of data.

- Used by a single team like Sales or HR
- Contains filtered and focused data
- Easier and faster for specific analysis

Example

Sales team stores only sales reports and customer purchases.

2. Data Warehouse

A Data Warehouse stores structured and cleaned data from multiple sources for reporting and analytics.

- Centralized storage
- Optimized for OLAP queries
- Used for business intelligence

Example

Company combines sales, finance, and customer data for company-wide analysis.

3. Data Lake

A Data Lake stores raw data in any format:

- Structured
- Semi-structured
- Unstructured
- Very scalable

- Used for big data and ML

Example

Stores raw logs, images, videos, JSON files, and sensor data together.

4. Data Lakehouse

A Data Lakehouse combines features of both:

- Data Lake flexibility
- Data Warehouse performance and management

Example

A company stores raw ML data and also runs fast SQL analytics on the same platform.

Connected Example

Imagine an e-commerce company:

Step 1 — Data Lake

All raw data enters the data lake:

- Website logs
- Product images
- Customer clicks
- Payment JSON data

Step 2 — Data Warehouse

Cleaned and structured sales/customer data is moved to the warehouse for reporting.

Step 3 — Data Mart

Sales department gets its own smaller sales-focused data mart.

Step 4 — Data Lakehouse

Instead of separate lake + warehouse systems, the company uses a lakehouse to handle both raw data and analytics together.

✓ **DATA COLD STORAGE**

Data cold storage is used to store data that is rarely accessed but must be kept for long periods.

Features

- Low storage cost
- Slower access speed
- Used for backup and archival purposes

Use Cases

- Old company records
- Backup files
- Compliance and legal data
- Historical logs

Examples

- [Amazon S3 Glacier](#)
- [Google Cloud Archive Storage](#)
- [Azure Archive Storage](#)

✓ **WHY DATA VERSIONING?**

If we don't version the data:

- Old data gets overwritten
- Changes cannot be tracked
- Difficult to reproduce ML model results
- Debugging becomes harder

- Risk of using wrong or inconsistent data
- Team collaboration becomes confusing

Importance of Data Versioning

Data versioning helps track different versions of datasets so experiments, reports, and models remain reliable and reproducible.

✓ RAID

RAID (Redundant Array of Independent Disks) combines multiple disks to improve speed, storage, and data protection.

Simple Example

Imagine you have 2 notebooks for storing class notes.

- If one notebook gets lost, the second notebook still has the notes.
- Or you can divide notes between notebooks to write faster.

RAID works similarly with hard disks.

RAID 0

- Fast performance
- No fault tolerance
- Uses data striping

RAID 1

- Data mirroring
- High safety
- More storage cost

RAID 5

- Good speed and fault tolerance
- Uses parity for recovery
- Common in servers

✓ WAYS TO OPTIMIZE DATABASE READS

- **Indexing** → Speeds up searching of data
- **Caching** → Stores frequently used data in memory
- **Read Replicas** → Multiple databases handle read requests
- **Query Optimization** → Write efficient SQL queries
- **Partitioning/Sharding** → Split large tables/databases
- **Proper Schema Design** → Organize tables efficiently
- **Limit Returned Data** → Select only needed columns/rows
- **Use Materialized Views** → Store precomputed query results

Simple Example

Adding an index to a “user_id” column helps the database find users much faster instead of scanning the whole table.

✓ REDIS

[Redis](#) is commonly used in ML deployments as a fast in-memory database/cache.

Uses of Redis in ML Deployments

- **Caching predictions** → Store frequent model outputs for faster response
- **Feature store caching** → Quickly access ML features
- **Session management** → Store user/session data
- **Task queues** → Handle async ML jobs using tools like Celery
- **Rate limiting** → Control API request load
- **Real-time data streaming** → Process live ML data quickly

Example

If many users request the same prediction, Redis stores the result temporarily so the model doesn't need to recompute it every time.

✓ **FEATURE STORE**

A Feature Store is a system used to store, manage, and serve ML features for training and prediction.

Raw Data Source

Raw data sources contain original unprocessed data from databases, logs, APIs, sensors, etc.

Difference

Feature Store	Raw Data Source
Stores processed ML features	Stores raw original data
Ready for ML models	Needs cleaning and processing
Fast and reusable	Large and unstructured

Simple Example

Raw Data

- Customer purchases
- Website clicks
- Login logs

Feature Store

Processed features like:

- Total purchases in last 30 days
- Average spending
- Login frequency

These ready-made features are directly used by ML models.

✓ **ETL PIPELINE**

ETL stands for:

- **Extract** → Collect data
- **Transform** → Clean/process data
- **Load** → Store data into a database or warehouse

Example

A company collects sales data from websites, cleans incorrect entries, and stores it in a data warehouse for reports.

✓ **AIRFLOW**

[Apache Airflow](#) is used to automate and schedule ETL pipelines.

Benefits

- Runs tasks automatically
- Manages workflow order
- Handles failures and retries
- Monitors pipelines easily

Simple Example

Every night Airflow automatically:

1. Collects sales data
2. Cleans the data
3. Loads it into the warehouse

4. Generates reports

✓ VECTOR DATABASE

A Vector Database stores and searches vector embeddings generated by AI/ML models. It is mainly used for similarity search in AI applications.

Why is it used?

Traditional databases search exact values, while vector DBs search similar meanings or patterns.

Real-life Example

In a chatbot, user questions are converted into vectors.

The vector DB finds the most similar previous documents or answers.

Differences Between Vector DB Offerings

Feature	Difference
Scalability	Some handle billions of vectors better
Speed	Search performance varies
Cloud Support	Some are fully managed cloud services
Filtering	Metadata filtering capabilities differ
Integration	Support for LangChain, LLMs, APIs differs

Examples of Vector DBs

- [Pinecone](#) → Managed cloud vector DB
- [Weaviate](#) → Open-source with ML integrations
- [Milvus](#) → High-scale vector search
- [Chroma](#) → Simple local AI applications

✓ LOG STORES

Log stores are systems used to collect, store, and manage application or system logs.

Logs contain records of events such as:

- Errors
- Requests
- System activity
- User actions

Significance in Observability

Log stores help monitor and debug systems by showing what happened inside applications and servers.

They help in:

- Detecting errors
- Troubleshooting failures
- Monitoring system health
- Security auditing

Real-life Example

If a website crashes, engineers check logs in the log store to find the exact error and time of failure.

Examples

- [Elasticsearch](#)

- [Splunk](#)
- [Grafana Loki](#)

FEATURE STORES (FEAST)

✓ CORE REQUIREMENT FOR EMERGENCE OF FEATURE STORE

The main requirement for a feature store is the need for **consistent and reusable ML features** across training and production systems.

Without a feature store:

- Teams repeatedly create the same features
- Training and serving data may differ
- ML pipelines become hard to manage

Real-life Example

A fraud detection system uses features like:

- Average transaction amount
- Number of transactions per day

A feature store ensures the exact same features are used:

- During model training
- During real-time prediction

This improves consistency and reduces errors.

✓ WHY ARE FEATURE STORES USEFUL IN SERVING/INFERENCE?

Feature stores help provide the same processed features used during training to the model during real-time prediction.

This ensures:

- Consistent features
- Faster inference
- Reduced training-serving mismatch

Real-life Example

A recommendation system uses features like:

- User's recent clicks
- Average watch time
- Purchase history

During inference, the feature store quickly serves these latest features to the model so it can make accurate recommendations in real time.

Important Point

Without a feature store, features may be computed differently in production, causing inaccurate predictions.

✓ ONLINE vs OFFLINE FEATURE STORE

Online Feature Store (Feast)

The online store is used for **real-time inference/predictions**.

- Fast access
- Stores latest feature values
- Optimized for low latency

Example

A fraud detection model instantly fetches a user's latest transaction count during payment processing.

Offline Feature Store (Feast)

The offline store is used for **training and batch analytics**.

- Stores historical feature data
- Handles large datasets
- Used for model training

Example

A recommendation model trains on months of user activity history stored offline.

Main Difference

Online Store	Offline Store
Real-time serving	Model training
Low latency	Large historical data
Latest features	Historical features

✓ LINEAGE IN A FEATURE STORE

Lineage means tracking where features come from, how they were created, and how they are used in ML pipelines.

It records:

- Source data
- Feature transformations
- Models using the features

Why is it Important?

- Helps debug feature issues
- Improves transparency
- Ensures reproducibility
- Tracks feature dependencies

Real-life Example

A fraud detection feature:

“Average transactions per day”

Lineage helps track:

- Which raw transaction data was used
- What calculations were applied
- Which ML models used this feature.

✓ HOW FEATURE STORES HELP MULTIPLE ML TEAMS

Feature stores allow multiple ML teams to share and reuse common features instead of creating them repeatedly.

Benefits

- Reduces duplicate work
- Maintains feature consistency
- Speeds up ML development
- Improves collaboration
- Ensures same features in training and inference

Real-life Example

Both fraud detection and recommendation teams may need:

- User purchase count
- Average spending

Instead of each team creating these separately, the feature store provides shared reusable features for all teams.

✓ **HALLUCINATION IN LLM**

Hallucination happens when a Large Language Model (LLM) generates incorrect, false, or made-up information while sounding confident.

The model produces answers that look believable but are not actually true.

Why Does it Happen?

- LLM predicts likely words, not facts
- Limited or outdated training data
- Missing context or unclear prompts

Important Point

Hallucination is a major challenge in AI systems, especially in medical, legal, and financial applications.

✓ **WHY DOES AN LLM GENERATE DIFFERENT RESPONSES FOR THE SAME QUESTION?**

LLMs use probabilistic prediction to generate text. They predict the next word based on probabilities, not fixed answers. Because of randomness in token selection, the same question can produce different responses.

Factors Affecting Responses

- Temperature setting
- Random sampling
- Prompt wording
- Context/history

✓ **RESPONSE PERSONALIZATION USING RAG WITH LLM**

In RAG (Retrieval-Augmented Generation), personalized information is retrieved from user-specific data and provided to the LLM before generating the response.

This helps the LLM generate responses tailored to the user.

How It Works

1. User query is received
2. Relevant user data/documents are retrieved
3. Retrieved context is sent to the LLM
4. LLM generates a personalized response

Real-life Example

In an e-learning app:

User asks:

“What should I study next?”

RAG retrieves:

- User’s completed courses
- Weak topics
- Previous scores

The LLM then gives a personalized study recommendation.

Benefits

- More relevant responses
- Better user experience
- Reduced hallucination
- Context-aware answers

CI/CD (GITHUB)

✓ **CONTINUOUS INTEGRATION (CI)**

Continuous Integration (CI) is a software development practice where developers regularly merge their code into a shared repository.
Whenever new code is added, automated tests and checks run to detect errors early.

Simple Example

A developer pushes new code to GitHub.
CI tools automatically build the project and run tests to check if the new code breaks anything.

✓ **CONTINUOUS DEPLOYMENT/DELIVERY (CD)**

CD is the process of automatically releasing tested code to staging or production after Continuous Integration (CI).

- **Continuous Delivery** → Code is ready for release, but deployment may need manual approval.
- **Continuous Deployment** → Code is automatically deployed without manual steps.
-

Simple Example

After tests pass in CI, a website update is automatically deployed to the live server using CD.

Benefits

- Faster releases
- Less manual work
- Quick bug fixes
- Reliable deployments

✓ **ROLE OF AUTOMATION IN CI/CD**

Automation is the core of CI/CD because it automatically handles building, testing, and deploying software without manual work.

It helps:

- Detect errors early
- Reduce human mistakes
- Speed up releases
- Ensure consistent deployments

Simple Example

When a developer pushes code to GitHub:

1. Tests run automatically
2. Application builds automatically
3. Deployment happens automatically if tests pass

This makes software delivery faster and more reliable.

✓ **ROADBLOCKS RESOLVED BY CONTINUOUS INTEGRATION (CI)**

Continuous Integration helps solve common software development problems such as:

- Code conflicts between developers

- Late bug detection
- Manual testing effort
- Integration issues
- Unstable builds
- Slow development cycles

Simple Example

Without CI, developers may combine code after many days and face large conflicts. With CI, code is integrated and tested regularly, so issues are found early and fixed quickly.

✓ ARCHITECTURE BEHIND GITHUB ACTIONS/WORKFLOWS

[GitHub Actions](#) works using event-driven workflows. When an event happens in a GitHub repository (like push, pull request, or release), a workflow is triggered automatically.

Main Components

- **Workflow** → Automation process written in YAML
- **Event Trigger** → Action that starts the workflow
- **Job** → Group of tasks
- **Step** → Individual command/task
- **Runner** → Machine that executes the workflow

Simple Flow

Developer pushes code → GitHub triggers workflow → Runner starts → Tests/build/deployment execute automatically.

Real-life Example

A developer pushes new API code:

1. Workflow starts
2. Tests run automatically
3. Docker image is built
4. Application is deployed to server/cloud.

✓ WHAT IS A ROLLBACK?

Rollback means reverting an application to a previous stable version when a new deployment causes problems.

Example

After deploying a new update, the website crashes. The system rolls back to the older working version.

✓ TYPES OF DEPLOYMENT

1. Blue-Green Deployment

Two environments are maintained. One runs the current version, the other receives the new version.

2. Canary Deployment

New version is released to a small group of users first.

3. Rolling Deployment

Application is updated gradually server by server.

4. Recreate Deployment

Old version is stopped completely before deploying the new one.

✓ **PYTEST**

[Pytest](#) is a Python testing framework used to write and run automated tests.

✓ **WORKFLOW TRIGGERS IN GITHUB ACTIONS**

Workflows are triggered automatically when specific events occur.

Types of Triggers

- push
- pull_request
- schedule (cron jobs)
- workflow_dispatch (manual trigger)
- release
- issue events

Example

When code is pushed to GitHub, a workflow automatically starts testing and deployment.

✓ **HOW IS THE VALIDATION SET USED IN CI?**

In CI, the validation set is used to test ML model performance after new code or model changes.

It helps verify that:

- Accuracy is acceptable
- Model quality has not degraded
- New updates do not break performance

Example

After training a new fraud detection model, CI automatically checks accuracy on the validation dataset before deployment.

✓ **BEST PRACTICES FOR CI**

- Commit small and frequent changes
- Automate testing and builds
- Run tests on every pull request
- Keep builds fast
- Use version control properly
- Maintain separate environments
- Monitor build failures quickly

Example

Whenever a developer pushes code, automated tests run immediately to catch issues early.

✓ **ALTERNATIVES TO GITHUB CI/CD ORCHESTRATION**

Other CI/CD tools include:

- [Jenkins](#)
- [GitLab CI/CD](#)
- [CircleCI](#)
- [Travis CI](#)
- [Azure DevOps](#)
- [Argo CD](#)

Example

A company may use Jenkins to automate testing, Docker builds, and Kubernetes deployments.

✓ **DIFFERENCE BETWEEN MERGING VS REBASING**

Merging

Combines branches while preserving complete history.

Rebasing

Moves branch commits on top of another branch to create a cleaner linear history.

Example

Feature branch updates from main:

- Merge → Keeps branch history
- Rebase → Rewrites history as if changes were made after latest main branch

✓ **MERGE CONFLICTS**

Merge conflicts happen when two branches modify the same part of a file differently and Git cannot decide which change to keep.

Example

Two developers edit the same line in app.py.

✓ **TYPES OF MERGING STRATEGIES**

Fast-Forward Merge

Branch moves forward without creating extra merge commit.

Three-Way Merge

Creates a new merge commit combining both histories.

Squash Merge

Combines all feature branch commits into one commit before merging.

Rebase and Merge

Replays commits on top of the target branch for cleaner history.

✓ **HOW DOES GIT KNOW ABOUT FILE CHANGES?**

Git uses hashing and compares snapshots of files between commits.

It tracks:

- Added files
- Modified files
- Deleted files

Example

If a line changes in a Python file, Git detects the file content difference using hashes.

✓ **DIFFERENCE BETWEEN STAGE AND COMMIT**

Stage

Temporary area where selected changes are prepared for commit.

Commit

Permanent snapshot of staged changes saved in Git history.

Example

- git add → Moves changes to staging area
- git commit → Saves changes permanently in repository history

✓ AUTHENTICATION VIA SSH IN GIT

SSH authentication allows secure communication between your computer and Git platforms without entering passwords repeatedly.

It uses:

- Public key
- Private key

✓ RUNNER IN CI

A runner is the machine or environment that executes CI/CD jobs and workflows.

Example

In GitHub Actions, the runner:

1. Pulls the code
2. Runs tests
3. Builds the application
4. Deploys the project.

MLFLOW

✓ **STAGING ENVIRONMENT**

Staging is a pre-production environment that closely matches the real production system. It is used to test the application before releasing it to actual users. In staging, teams validate whether the application works correctly in a production-like setup.

Common validations include:

- Functional testing
- Integration testing
- API testing
- UI/UX validation
- Performance testing
- Security checks
- Database migration testing

Real-life Example

Before launching a new payment feature on an e-commerce website, the feature is first tested in staging to ensure payments, APIs, and user flows work correctly without affecting real customers.

✓ **CODE LIFECYCLE PATTERNS**

Code lifecycle patterns are the stages and practices used to develop, test, release, and maintain software safely and efficiently.

They define how code moves from development to production.

Common Stages

- Development
- Testing
- Integration
- Staging
- Deployment
- Monitoring
- Rollback/Updates

Purpose

They help:

- Reduce bugs
- Improve software quality
- Automate releases
- Ensure reliable deployments

✓ **CAN WE APPLY DEPLOYMENT LIFECYCLE PATTERNS TO PRE-TRAINING LLMS?**

Yes. Software deployment lifecycle patterns are also important in pre-training and managing Large Language Models (LLMs).

LLM training pipelines involve:

- Data collection
- Data validation
- Model training
- Evaluation
- Testing
- Deployment
- Monitoring

Practices like CI/CD, versioning, staging, rollback, and automation help manage these complex workflows reliably.

Real-life Example

Before deploying a newly trained LLM:

1. The model is tested on validation benchmarks
2. Safety and hallucination checks are performed
3. The model is deployed to staging
4. If issues occur, rollback restores the previous stable model version

This ensures stable and reliable AI model releases.

✓ **INDUSTRY COMPLIANCES IN FINANCIAL AND HEALTHCARE DOMAINS**

Industry compliances are rules and regulations that ensure data security, privacy, and proper system usage.

In finance, compliances protect banking and payment data.

In healthcare, compliances protect patient and medical information.

Examples

- [PCI DSS](#) → Payment card security
- [HIPAA](#) → Patient health data protection
- [GDPR](#) → User data privacy regulation

Real-life Example

A hospital must protect patient medical records and allow access only to authorized staff.

✓ **WHY IS PRODUCTION DATA ACCESS RESTRICTED IN ML WORKFLOWS?**

Production data may contain sensitive or confidential information.

Direct access is restricted to protect:

- User privacy
- Security
- Regulatory compliance
- Business-critical information

It also prevents accidental data modification or leakage.

Real-life Example

An ML engineer training a recommendation model may use anonymized or sample data instead of real customer payment information from production systems.

✓ **HIPAA**

[HIPAA](#) (Health Insurance Portability and Accountability Act) is a US healthcare law that protects sensitive patient health information.

It ensures:

- Patient data privacy
- Secure medical records
- Controlled access to healthcare information

Real-life Example

A hospital must securely store patient medical records and prevent unauthorized access.

✓ **GDPR**

[GDPR](#) (General Data Protection Regulation) is a European data privacy law that protects personal user data.

It gives users rights over their data, such as:

- Accessing their data
- Deleting their data
- Controlling data usage

Real-life Example

A website must ask users for cookie/data consent before collecting personal information.

✓ **WHAT ARE COMPLIANCE CHECKS?**

Compliance checks are validations performed to ensure systems, data, and processes follow legal, security, and industry regulations.

They verify:

- Data privacy
- Security standards
- Access controls
- Proper data handling

Why Are They Important?

Compliance checks help:

- Protect sensitive data
- Prevent security breaches
- Avoid legal issues
- Build user trust

Real-life Example

A banking application checks whether customer payment data is encrypted according to PCI DSS rules.

✓ **WHAT HAPPENS IF COMPLIANCE CHECKS FAIL?**

If compliance checks fail:

- Deployment may be blocked
- Systems may face legal penalties
- Data breaches may occur
- Company reputation can be damaged
- Access to production systems may be restricted

Real-life Example

If a healthcare application stores patient records insecurely, it may violate HIPAA regulations and face fines or legal action.

✓ **HOW DO YOU COMPARE MODELS WHILE ITERATING OVER PARAMETERS?**

Models are compared using evaluation metrics on validation data after changing parameters.

Common metrics include: Accuracy, Precision, Recall, F1-score & Loss

The model with better performance and generalization is selected.

Real-life Example

A spam classifier is trained with different parameter values, and the model with highest validation accuracy is chosen.

✓ **HYPERPARAMETER TUNING**

Hyperparameter tuning is the process of finding the best hyperparameter values to improve model performance. Hyperparameters are settings chosen before training starts.

Examples

- Learning rate
- Batch size
- Number of trees
- Epochs

Real-life Example

Trying different learning rates to find which gives the best prediction accuracy.

Why Do We Need Hyperparameters?

Hyperparameters control how a model learns.

Without proper hyperparameters:

- Training may be too slow
- Model may overfit or underfit
- Accuracy may decrease

Different datasets require different learning behavior.

Why Not Train Without Them?

ML algorithms need configuration settings to guide optimization and learning processes.

Real-life Example

If learning rate is too high, a model may learn unstable patterns and fail to converge properly.

✓ **HOW DOES BAYESIAN OPTIMIZATION WORK?**

Bayesian optimization finds the best hyperparameters by learning from previous training results instead of trying random values blindly. It predicts which parameter combinations are most likely to improve model performance and tests those next.

Real-life Example

If a model performs well with a learning rate of 0.01, Bayesian optimization intelligently searches nearby values instead of testing random values everywhere.

✓ **WHAT ARE EVOLUTIONARY ALGORITHMS FOR HYPERPARAMETER TUNING?**

Evolutionary algorithms optimize hyperparameters using concepts inspired by natural evolution such as:

- Selection
- Mutation
- Crossover

Better parameter combinations survive and generate new combinations over multiple iterations.

Real-life Example

Different sets of hyperparameters compete like “survival of the fittest,” and the best-performing combinations are retained for future training rounds.

✓ **ALTERNATIVES TO MLFLOW**

[MLflow](#) alternatives include:

- [Weights & Biases](#)
- [Neptune.ai](#)
- [Kubeflow](#)
- [Comet](#)
- [ClearML](#)

These tools help with:

- Experiment tracking
- Model versioning
- Hyperparameter logging
- ML workflow management.

DOCKER & KUBERNETES

✓ **HORIZONTAL SCALING**

Horizontal scaling is the process of increasing system capacity by adding multiple servers or instances to distribute workload efficiently.

✓ **VERTICAL SCALING**

Vertical scaling is the process of improving system performance by upgrading the hardware or resources of an existing server.

✓ **HOW DOCKER HELPS ML TEAMS SHIP FASTER**

Docker enables ML teams to create portable and consistent environments, reducing deployment issues and accelerating development and release cycles.

Docker packages the entire application along with:

- Code
- Libraries
- Dependencies
- Python version
- Environment settings

into one container.

This ensures the application runs the same everywhere.

✓ **HOW FASTAPI SCALES**

FastAPI can utilize multicore processing through multiple worker processes, enabling parallel request handling across CPU cores.

FastAPI can scale using:

- Multiple Docker containers
- Load balancers
- Kubernetes
- Cloud auto-scaling
- Async APIs

✓ **GLOBAL INTERPRETER LOCK (GIL)**

The Global Interpreter Lock (GIL) is a mechanism in Python that allows **only one thread to execute Python bytecode at a time** inside a single process.

✓ **PYTHON vs NODEJS**

Python provides robust multiprocessing support suitable for CPU-intensive and machine learning workloads, while Node.js excels in asynchronous, event-driven concurrency for high-throughput I/O applications.

✓ **KUBERNETES**

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, networking, and management of containerized applications.

Simple Example

Imagine managing 100 food delivery riders manually.

Very difficult.

Kubernetes acts like a smart manager that:

- Assigns work

- Replaces inactive riders
- Balances workload
- Ensures deliveries continue smoothly

✓ **CONTAINER ORCHESTRATION**

Container orchestration is the automated management, deployment, scaling, and coordination of containerized applications.

Simple Example

Instead of manually controlling every traffic signal in a city, an automated traffic system controls everything intelligently. That automation is orchestration.

✓ **POD vs CONTAINER**

A Kubernetes pod is the smallest deployable unit in Kubernetes that can contain one or more containers sharing networking and storage resources.

Real-Life Example

Think of:

- Container = passenger
- Pod = car carrying passengers

The pod provides:

- Networking
- Shared storage
- Communication environment

✓ **DEPLOYMENT vs SERVICE**

Deployment

A deployment manages:

- Pods
- Scaling
- Updates
- Replica count

It ensures the application keeps running.

Service

A service provides:

- Stable networking
- Access to pods
- Load balancing

It exposes the application internally or externally.

Simple Example

A deployment is like managing workers in a company.

A service is like the reception desk that directs customers to available workers.

✓ **NODE IN K8S**

A node is a worker machine in a Kubernetes cluster responsible for running containerized workloads.

Simple Example

If Kubernetes is a school:

- Pods are students
- Nodes are classrooms

Pods run inside nodes.

✓ **HOW DOES KUBERNETES HANDLE NODE FAILURES?**

Kubernetes handles node failures through self-healing mechanisms that automatically reschedule workloads onto healthy nodes.

If a node fails:

- Pods on that node become unavailable
- Kubernetes automatically creates replacement pods on healthy nodes

Simple Example

If one delivery hub shuts down, deliveries are redirected to another nearby hub automatically.

Important Benefit

Applications remain available with minimal downtime.

✓ **ARTIFACT REGISTRY**

An artifact registry stores:

- Docker images
- Application artifacts

If the Registry Fails

New pods may fail to start because Kubernetes cannot pull container images.

However:

- Already running containers usually continue running
- Existing pods are not immediately affected

Common Industry Solutions

Teams use:

- Multiple registries
- Cached images
- Backup registries
- High availability setups

DOCKER vs KUBERNETES

[Docker](#) is used to **package and run** an application inside a container.

Kubernetes is used to **manage many containers automatically** at large scale.

FEDERATED LEARNING

Federated learning is a machine learning approach where:

- Multiple devices or organizations train a model locally
- Raw data never leaves their systems
- Only model updates are shared and combined

This improves:

- Privacy
- Security
- Distributed training

HOW GOOGLE ADS WORK TO CAPTURE INTERESTS WITHOUT SENDING DATA TO EXTERNAL SERVERS

On-Device Interest Extraction in Android Advertising Systems

Modern Android advertising systems increasingly rely on on-device processing to improve user privacy while still enabling personalized advertisements. Instead of continuously sending raw user activity to external servers, the device itself performs local analysis using machine learning models.

The system observes behavioral signals such as:

- Application usage patterns
- Content interaction categories
- Ad engagement frequency
- General activity trends

Using these signals, the device generates broad interest categories like technology, gaming, travel, or fitness. This process happens locally on the phone, reducing the need to transmit sensitive raw behavioral data externally.

Rather than sharing complete user activity logs, the system typically sends limited advertising signals such as:

- Aggregated interests
- Anonymous identifiers
- Temporary advertising tokens

This allows ad platforms to display relevant advertisements without directly accessing detailed personal information.

Modern Android privacy systems also integrate technologies such as federated learning, differential privacy, and secure aggregation. These techniques help improve personalization while minimizing direct exposure of user data.

An important concept is that on-device processing does not completely eliminate external communication. Some information may still be transmitted for:

- Ad delivery
- Fraud detection
- Analytics
- Performance measurement

However, the primary objective is to reduce large-scale behavioral tracking and increase privacy-preserving computation on the user's device.

In professional terms, Android's advertising ecosystem uses on-device machine learning and privacy-focused architectures to infer user interests locally while limiting the transmission of raw personal data to external systems.